

ui.date_input 全面详解

`ui.date_input` 是 NiceGUI 3.3.0 版本新增的日期输入组件，基于 Quasar 的 `QInput` 组件扩展，集成了日期选择器功能，支持单日期选择、日期范围选择、日期过滤等核心能力，同时提供灵活的样式自定义和数据绑定支持，适用于表单日期录入、时间范围筛选等场景（如预约系统、数据查询界面、任务截止日期设置等）。

一、核心初始化参数

初始化 `ui.date_input` 时可配置 5 个关键参数，覆盖基础功能需求，参数设计简洁且实用性强：

参数名	说明
<code>label</code>	组件的显示标签，用于提示用户输入用途（如 "预约日期"、"生效日期"）
<code>range_input</code>	布尔值，是否启用日期范围选择（默认 <code>False</code> ）： - <code>False</code> ：单日期选择，值为字符串格式（如 <code>2025-05-31</code> ） - <code>True</code> ：日期范围选择，值为 <code>startdate - enddate</code> 格式字符串（如 <code>2025-05-01 - 2025-05-31</code> ）
<code>placeholder</code>	未选择日期时显示的占位文本（如 "请选择日期"、"选择开始 - 结束日期"）
<code>value</code>	初始日期值： - 单日期模式：直接传入日期字符串（如 <code>2025-05-31</code> ） - 范围模式：传入 <code>start-end</code> 格式字符串（如 <code>2025-05-01 - 2025-05-31</code> ）
<code>on_change</code>	日期值变更时触发的回调函数，事件对象通过 <code>e.value</code> 获取当前选中日期 / 范围

基础使用示例

1. 单日期选择（默认模式）

```
from nicegui import ui

# 初始化单日期选择器，初始值为 2025-05-31
date_picker = ui.date_input(
    label='任务截止日期',
    placeholder='请选择截止日期',
    value='2025-05-31',
    on_change=lambda e: ui.notify(f'截止日期已设置为: {e.value}')
)

# 实时显示当前选中日期（通过数据绑定）
ui.label().bind_text_from(date_picker, 'value', lambda v: f'当前选择: {v}')

ui.run()
```

2. 日期范围选择（启用 range_input=True）

```
from nicegui import ui

# 初始化日期范围选择器
range_picker = ui.date_input(
    label='查询时间范围',
    placeholder='选择开始-结束日期',
    value='2025-05-01 - 2025-05-31',
    range_input=True,
    on_change=lambda e: ui.notify(f'查询范围: {e.value}')
)
range_picker.classes('w-80') # 调整组件宽度

# 实时显示选中范围
ui.label().bind_text_from(range_picker, 'value', lambda v: f'当前范围: {v}')

ui.run()
```

二、组件核心属性

`ui.date_input` 继承 NiceGUI 基础元素的核心属性，支持动态状态控制、样式自定义、DOM 标识等能力，关键属性如下：

属性名	类型	说明	
<code>classes</code>	<code>Classes[Self]</code>	组件的 HTML 类名，用于通过 Tailwind/Quasar 样式自定义外观（如宽度、边框、间距）	
<code>client</code>	<code>Client</code>	组件所属的客户端实例（多客户端场景下的隔离标识）	
<code>enabled</code>	<code>BindableProperty</code>	组件是否启用（可绑定， <code>False</code> 时禁止用户交互，支持动态启用 / 禁用）	
<code>html_id</code>	<code>str</code>	组件在 HTML DOM 中的唯一 ID（2.16.0 版本新增，用于精准 DOM 操作）	
<code>is_deleted</code>	<code>bool</code>	组件是否已被删除（只读属性，用于判断组件生命周期状态）	
<code>is_ignoring_events</code>	<code>bool</code>	组件是否忽略事件（只读属性，用于调试或事件控制场景）	
<code>label</code>	<code>BindableProperty</code>	组件标签（支持动态绑定，可通过 <code>set_label</code> 方法修改）	

属性名	类型	说明	
<code>parent_slot</code>	<code>`Slot`</code>	<code>None`</code>	组件的父插槽 (可设置, 用于复杂布局中的插槽嵌套)
<code>props</code>	<code>Props[Self]</code>	组件的 Quasar 原生属性 (用于扩展输入框本身的行为, 如 <code>dense</code> 紧凑模式、 <code>readonly</code> 只读模式)	
<code>picker</code>	<code>ui.date</code> 实例	底层日期选择器对象, 用于自定义日期选择规则 (如日期过滤、视图配置)	
<code>style</code>	<code>Style[Self]</code>	组件的内联 CSS 样式 (如 <code>width: 300px; margin: 10px 0</code>)	
<code>value</code>	<code>BindableProperty</code>	当前选中的日期 / 范围值 (支持动态绑定, 可通过 <code>set_value</code> 方法修改)	
<code>visible</code>	<code>BindableProperty</code>	组件是否可见 (支持动态绑定, <code>False</code> 时隐藏组件)	

属性操作示例

```
from nicegui import ui

# 初始化单日期选择器
date_picker = ui.date_input(label='初始标签', value='2025-05-31')

# 按钮控制组件状态
ui.button('禁用选择器', on_click=date_picker.disable)
ui.button('启用选择器', on_click=date_picker.enable)
ui.button('修改标签', on_click=lambda: date_picker.set_label('更新后的日期标签'))
ui.button('设置为今天', on_click=lambda: date_picker.set_value('2025-06-15'))
ui.button('隐藏选择器', on_click=lambda: date_picker.set_visibility(False))

ui.run()
```

三、核心特性：日期过滤与选择器自定义（通过 `picker` 属性）

`ui.date_input` 的底层日期选择器由 `ui.date` 组件实现, 可通过 `picker` 属性访问并自定义其行为, 核心应用场景是**日期过滤** (如限制可选日期范围、禁用过去日期等) 。

常用 picker 自定义场景

1. 限制可选日期范围（仅允许选择 2025-11-10 及以后的日期）

```
from nicegui import ui

date_picker = ui.date_input(
    label='预约日期',
    value='2025-11-15',
    placeholder='仅允许选择 2025-11-10 及以后的日期'
)

# 通过 picker.props 配置日期过滤规则（Vue 表达式）
# 规则：日期 >= "2025/11/10"（注意日期格式为 yyyy/mm/dd）
date_picker.picker.props[':options'] = 'date => date >= "2025/11/10"'

# 实时显示选中日期
ui.label().bind_text_from(date_picker, 'value', lambda v: f'已选择: {v}')

ui.run()
```

2. 禁用周末（仅允许选择工作日）

```
from nicegui import ui

date_picker = ui.date_input(label='工作日选择')

# 日期过滤规则：排除周六（6）和周日（0）
date_picker.picker.props[':options'] = 'date => date.getDay() !== 0 && date.getDay() !== 6'

ui.run()
```

picker 属性说明

- `picker` 是 `ui.date` 组件的实例，支持 `ui.date` 的所有配置（如 `props`、`style`、`classes` 等）；
- 日期过滤通过 `picker.props[':options']` 配置，值为 Vue 表达式（接收 `date` 参数，返回布尔值，`True` 表示日期可选）；
- 更多 `ui.date` 配置可参考 [NiceGUI ui.date 文档](#)。

四、核心方法

`ui.date_input` 提供丰富的方法用于组件状态控制、数据绑定、样式自定义等，按功能分类如下：

1. 状态控制方法

用于直接修改组件的启用状态、可见性、标签等基础属性：

方法名	说明	
<code>enable()</code>	启用组件（允许用户交互）	
<code>disable()</code>	禁用组件（禁止用户交互，输入框变灰）	
<code>set_enabled(value: bool)</code>	动态设置启用状态（ <code>True</code> 启用， <code>False</code> 禁用）	
<code>set_label(label: str</code>	<code>None)</code>	修改组件标签（传 <code>None</code> 隐藏标签）
<code>set_value(value: str)</code>	设置日期 / 范围值（单日期传日期字符串，范围传 <code>start-end</code> 字符串）	
<code>set_visibility(visible: bool)</code>	动态设置组件可见性（ <code>True</code> 显示， <code>False</code> 隐藏）	
<code>clear()</code>	清除所有子元素（极少用于日期输入框，适用于嵌套复杂内容场景）	
<code>delete()</code>	删除组件及其所有子元素（释放资源，生命周期结束）	

2. 数据绑定方法

支持将组件的 `enabled`、`label`、`value`、`visible` 等属性与目标对象属性进行单向 / 双向绑定，实现数据同步：

方法名	说明
<code>bind_enabled(...)</code>	双向绑定组件启用状态与目标对象属性
<code>bind_enabled_from(...)</code>	单向绑定（目标对象 → 组件）启用状态
<code>bind_enabled_to(...)</code>	单向绑定（组件 → 目标对象）启用状态
<code>bind_label(...)</code>	双向绑定组件标签与目标对象属性
<code>bind_label_from(...)</code>	单向绑定（目标对象 → 组件）标签
<code>bind_label_to(...)</code>	单向绑定（组件 → 目标对象）标签
<code>bind_value(...)</code>	双向绑定组件日期值与目标对象属性（核心绑定方法）
<code>bind_value_from(...)</code>	单向绑定（目标对象 → 组件）日期值
<code>bind_value_to(...)</code>	单向绑定（组件 → 目标对象）日期值
<code>bind_visibility(...)</code>	双向绑定组件可见性与目标对象属性

绑定示例（与数据类同步）

```
from dataclasses import dataclass
from nicegui import ui

# 定义数据类（用于存储业务数据）
@dataclass
class Task:
    deadline: str = '2025-05-31' # 与日期选择器绑定的属性

task = Task()

# 日期选择器与 task.deadline 双向绑定
date_picker = ui.date_input(label='任务截止日期').bind_value(task, 'deadline')

# 显示绑定的当前值
ui.label().bind_text_from(task, 'deadline', lambda v: f'任务截止日期: {v}')

# 按钮修改数据类属性（间接同步到日期选择器）
ui.button('延长截止日期', on_click=lambda: setattr(task, 'deadline', '2025-06-10'))

ui.run()
```

3. 样式与扩展方法

用于自定义组件外观、添加资源或辅助功能：

方法名	说明
<code>classes(add/remove/toggle/replace)</code>	新增 / 移除 / 切换 / 替换组件的 HTML 类（如 <code>date_picker.classes('w-60 border-red-500')</code> ）
<code>style(add/remove/replace)</code>	新增 / 移除 / 替换组件的内联 CSS 样式（如 <code>date_picker.style('width: 300px;')</code> ）
<code>default_classes(...)</code>	全局修改该类组件的默认 HTML 类（需在实例化前调用，如统一设置宽度）
<code>default_style(...)</code>	全局修改该类组件的默认 CSS 样式（需在实例化前调用）
<code>tooltip(text: str)</code>	为组件添加 tooltip 提示（鼠标悬浮时显示，如 <code>tooltip('选择任务截止日期')</code> ）
<code>add_resource(path)</code>	为组件添加资源文件（如自定义 CSS/JS，用于扩展样式或功能）
<code>mark(*markers)</code>	为组件添加标记（用于测试查询或依赖管理）

样式自定义示例

```
# 全局设置所有日期输入框的默认样式（实例化前调用）
ui.date_input.default_classes(add='shadow-sm border-gray-300 rounded-lg')
ui.date_input.default_style(add='margin: 8px 0; padding: 4px;')

# 实例化时添加局部样式
range_picker = ui.date_input(
    label='样式自定义示例',
    range_input=True,
    value='2025-05-01 - 2025-05-31'
)
range_picker.classes('w-96 border-blue-500') # 局部类
range_picker.style('font-size: 14px;') # 局部内联样式
range_picker.tooltip('选择时间范围') # 添加提示

ui.run()
```

4. 其他实用方法

方法名	说明
<code>ancestors(include_self)</code>	迭代组件的祖先元素（ <code>include_self=True</code> 包含自身）
<code>descendants(include_self)</code>	迭代组件的子元素（ <code>include_self=True</code> 包含自身）
<code>get_computed_prop(prop_name, timeout)</code>	获取计算属性（需异步等待，如获取组件实际渲染后的宽度）
<code>move(target_container, target_index, target_slot)</code>	移动组件到其他容器（用于动态布局调整）
<code>on(type, handler, ...)</code>	订阅通用 DOM 事件（如点击、鼠标悬浮等）
<code>on_value_change(callback)</code>	绑定日期值变更事件（与初始化 <code>on_change</code> 参数等价）
<code>remove(element)</code>	移除子元素（极少使用）
<code>run_method(name, *args)</code>	运行客户端方法（如调用底层 Quasar 组件的原生方法）
<code>update()</code>	强制在客户端更新组件状态（用于手动同步数据，如动态修改过滤规则后刷新）

五、事件处理

1. 核心事件：日期值变更（`on_change/on_value_change`）

日期值变更事件是 `ui.date_input` 的核心事件，支持两种绑定方式（功能等价）：

```
# 方式1: 初始化时通过 on_change 参数绑定
ui.date_input(
    label='事件绑定示例',
    on_change=lambda e: print(f'日期变更为: {e.value}')
)

# 方式2: 通过 on_value_change 方法绑定
date_picker = ui.date_input(label='事件绑定示例')
date_picker.on_value_change(lambda e: print(f'日期变更为: {e.value}'))
```

2. 通用事件订阅 (on 方法)

通过 `on()` 方法订阅其他 DOM 事件（如点击、鼠标悬浮、聚焦等），支持客户端 JS 处理或服务端 Python 处理：

```
from nicegui import ui

date_picker = ui.date_input(label='通用事件示例', value='2025-05-31')

# 订阅聚焦事件（服务端处理）
date_picker.on('focus', lambda: print('日期输入框获得焦点'))

# 订阅失去焦点事件（客户端 JS 处理）
date_picker.on(
    'blur',
    js_handler='(e) => console.log("日期输入框失去焦点, 当前值: " + e.target.value)'
)

ui.run()
```

六、版本兼容性说明

特性 / 属性	支持版本
<code>ui.date_input</code> 组件本身	3.3.0+（新增）
<code>html_id</code> 属性	2.16.0+
<code>toggle</code> 参数（ <code>default_classes</code> ）	2.7.0+
<code>strict</code> 参数（绑定方法）	3.0.0+
同时指定 Python 和 JS 事件处理（ <code>on</code> 方法）	2.18.0+

七、关键注意事项

1. 日期格式规范：
- 单日期模式和范围模式的初始值、设置值均需遵循 `yyyy-mm-dd` 格式（如 `2025-05-31`），否则可能导致组件渲染异常；
 - 日期过滤规则中，Vue 表达式内的日期格式需使用 `yyyy/mm/dd`（如 `date >= "2025/11/10"`）。

2. 范围值处理：

启用 `range_input=True` 后，`value` 为 `start-end` 格式字符串，如需拆分起始日期和结束日期，可通过字符串分割处理：

```
def handle_range_change(e):
    start_date, end_date = e.value.split(' - ')
    print(f'起始日期: {start_date}, 结束日期: {end_date}')

ui.date_input(range_input=True, on_change=handle_range_change)
```

3. picker 属性使用限制：

`picker` 是底层 `ui.date` 实例，修改其属性时需确保符合 `ui.date` 的配置规则（如 `props` 需传入 Vue 支持的表达式），否则可能导致日期选择器功能异常。

总结

`ui.date_input` 是 NiceGUI 3.3.0+ 版本推出的高效日期输入组件，核心优势在于：

1. 支持单日期 / 日期范围双模式，覆盖绝大多数日期输入场景；
2. 集成底层 `ui.date` 组件，支持灵活的日期过滤规则自定义；
3. 提供完善的数据绑定能力，可与业务数据模型无缝同步；
4. 样式自定义能力强，支持全局默认样式和局部样式调整；
5. 基于 Quasar 组件，交互流畅且跨浏览器兼容。

适用于表单录入、数据筛选、时间规划等场景，结合 NiceGUI 的按钮、标签、卡片等组件可快速构建直观的日期交互界面。